SMU Singapore Management University | School of Computing and Information Systems

Computational Thinking with Python

Jean Chen **James Koh**

{jeanchen, jameskohby}@smu.edu.sg

Computational Thinking

- Decomposition
- Pattern Recognition
- Abstraction
- Algorithmic Thinking

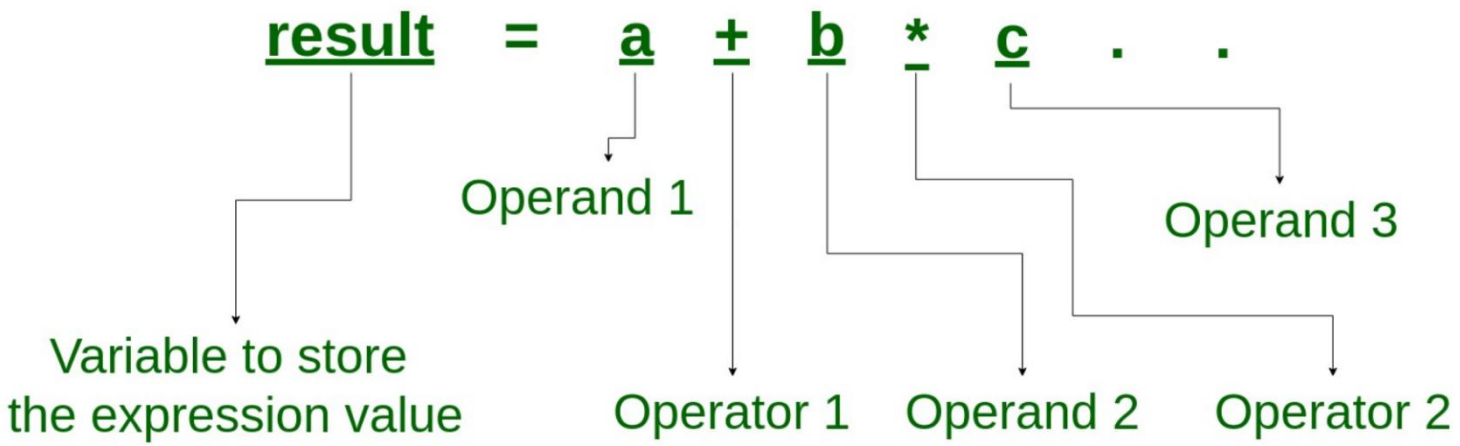
A function (programming) is essentially an abstraction.

© MITB, SMU

Functions

Operators and operands

- Operators → what action or operation to perform?
- Operands → what items to apply the action to?



Variables, expressions, operators and operands

geeksforgeeks.org/what-is-an-expression-and-what-are-the-types-of-expressions

© MITB, SMU

Operator examples

- Membership Operators (in / not in)
- **in** – Returns true if a variable/value found in the sequence.
- **not in** – Returns true if a variable/value not found in the sequence.

string, list, tuples, sets, dictionary…

```
str1 = 'abc'
str2 = 'def'
'a' in str1      #True
'a' in str2      #False
'a' not in str1  #False
'a' not in str2  #True
```


© MITB, SMU

Functions

- Functions act as operators.
  - Operators: more primitive, pre-defined by the programming language itself
  - Functions: self defined, or defined by a library
- A function encapsulates a block of code.
- A function is designed to perform a specific task or calculation.

Encapsulation

As a consequence,

- Functions are used to reduce repetitive code.
  - why?
- Functions are also used to structure a program so that code logic is clearer.
  - why?

Reusability

Readability

© MITB, SMU

Elements of Functions – Design

- Encapsulation
  - What does this function do?
  - What is the specific task?
  - What if this function is missing?
- Reusability
  - Multiplicity: to be used many times
  - Heterogeneity: to be used in different scenarios

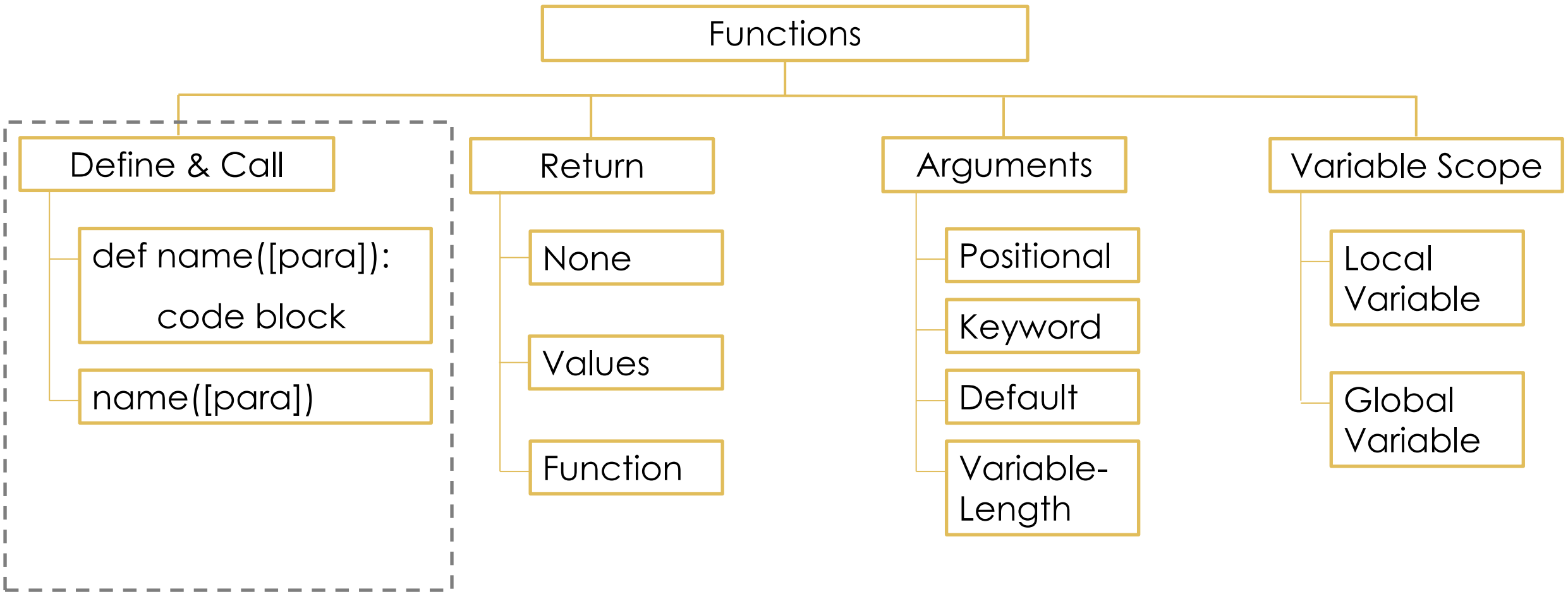
© MITB, SMU

Elements of Functions – Implementation

- Return
  - What is the result from this function?
  - Will the return value cause confusion?
- Arguments
  - What parameters to pass into this function?
  - Have I considered all scenarios when this function is to be called?

© MITB, SMU

Functions

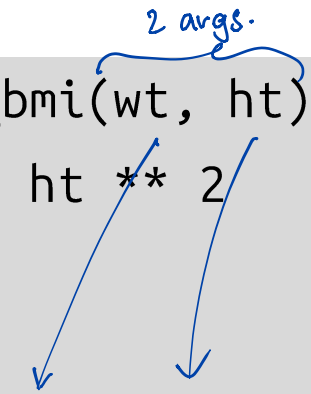


© MITB, SMU

Define & Call

- **def** tells python that this command is a definition of a function.
  - Usually associate the function to the task it completes.
- The code block processes the input and produces the output.
- Function name must start with a letter or an underscore.
  - Names may contain digits, but do not start with digits.

```
def function_name(argument list):
    code block
```

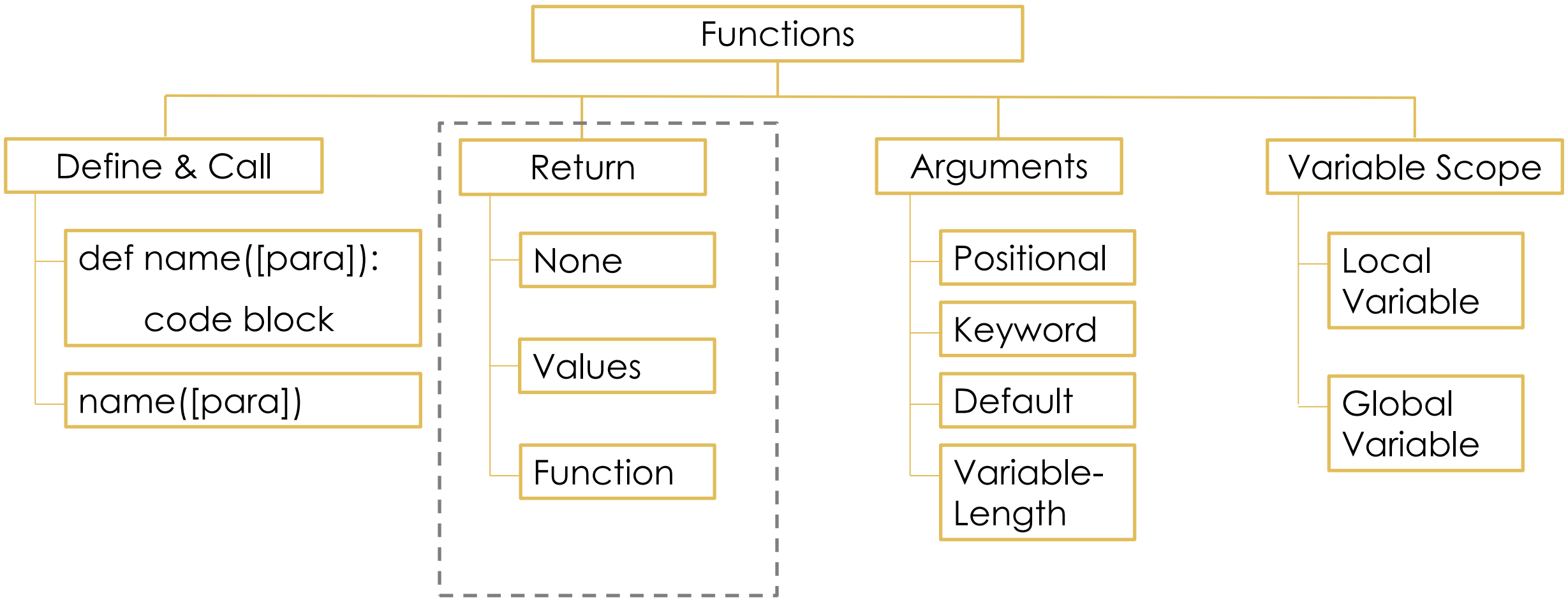



```
def calculate_bmi(wt, ht):
    bmi = wt / ht ** 2
    print(bmi)

calculate_bmi(45, 1.5)
# This is how we call the function
```


© MITB, SMU

Functions



© MITB, SMU

return

- A return command terminated the function execution.
- It returns the result to the caller.
- The statements after the return statements are not executed.

```
def func():
    code block

    return [expression]
```


© MITB, SMU

return None

- If nothing is after return or there is no return statement, then the special value None is returned.
- All three functions below return None.

```
def func1():
    result = 1

def func2():
    return

def func3():
    return None
```


© MITB, SMU

return Values

- It can return primitive data types, or containers.

```
def func1(a, b):
    return a

def func2(a, b):
    return a, b

func1(1, 3)   # output: 1
func2(1, 3)   # output: 1,3
```

```
def func3(a, b):
    return [a, b]

def func4(a, b):
    return {'a': a, 'b': b}

func3(1, 3)  # output: [1,3]
func4(1, 3)  # output: {'a':1,'b':3}
```


© MITB, SMU

return another function

- Function is an object that can be assigned to a variable, passed as an argument to a function, or used as a return value.

```
def scale_by_factor(factor):
    def multiply(number):
        return factor * number
    return multiply

double = scale_by_factor(2)    # now double is a function
double(2)                      # output: 4
double(3)                      # output: 6
triple = scale_by_factor(3)    # another function
```


© MITB, SMU

return – Example

Use return statements for assignments

- What is the difference between the two functions below?

```
def add1(a, b):
    c = a + b
    return c

result = add1(2, 3)
print(result)          # 5
```

```
def add2(a, b):
    c = a + b
    print(c)
    return None

result2 = add2(2, 3)   # 5
print(result2)         # None
```


© MITB, SMU

Wooclap!

What is the output of the below code?

```
def hello():
    print("1", end=" ")
    return None → evaluates to false (falsy)
if hello():
    print("2", end=" ")
else:
    print("3", end=" ")  ← goes here
```

a. Nothing is printed

b. 1 2

c. 1 3

d. 1 2 3

Example – Logistic Function

- Standard logistic function, commonly used in machine learning:

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Try to calculate *f(-100), f(0), f(100)* by python, is it somewhat troublesome without writing a function?

```
import math
def logistic(x):
    return 1/(1+math.exp(-x))

print(logistic(-100))
print(logistic(0))
print(logistic(100))
```


© MITB, SMU

Function examples

- Write a function to give the last day of a month,
  e.g., last_day_of_month(5) = 31, last_day_of_month(6) = 30, …

```
def last_day_of_month(month):
    if month == 2:
        return 28
    elif month <= 7 == month % 2:
        return 31
    else:
        return 30
```

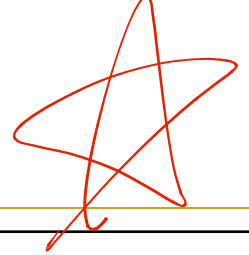


```
def last_day_of_month(month):
    if month == 2:
        return 28
    elif (month <= 7) == (month % 2):
        return 31
    else:
        return 30
```

- Now, execute, review and extend

© MITB, SMU

Operator precedence



- Operator precedence refers to the rules that determine the order in which different operators are evaluated in an expression.

- What is the order of operators in

```
month <= 7 == month % 2
```

- Is this the issue just now?

```
month <= 7 == 1
⇔ (month <= 7) and (7 == 1)
```

Operators	Meaning
`()`	Parentheses
`**`	Exponent
`+x, -x, ~x`	Unary plus, Unary minus, Bitwise NOT
`*, /, //, %`	Multiplication, Division, Floor division, Modulus
`+, -`	Addition, Subtraction
`<<, >>`	Bitwise shift operators
`&`	Bitwise AND
`^`	Bitwise XOR
`|`	Bitwise OR
`==, !=, >, >=, <, <=, is, is not, in, not in`	Comparisons, Identity, Membership operators
`not`	Logical NOT
`and`	Logical AND
`or`	Logical OR
`if-else`	Ternary conditional
`lambda`	Anonymous function
`=, +=, -=, *=, /=, //=, …`	(Compound) assignment

www.programiz.com/python-programming/precedence-associativity

© MITB, SMU

Function examples

- Check validity!

- How about the last day of February on leap years?

```
def last_day_of_month(month):
    if month <= 0 or month > 12:
        print('not a valid month')
        return -1
    if month == 2:
        return 28
    elif (month <= 7) == (month % 2):
        return 31
    else:
        return 30
```

2 possibilities only

1. month <= 7 == 0
2. month <= 7 == 1

© MITB, SMU

Function examples

- Write a function to tell if it is a leap year

```
def is_leap_year(year):
    if year % 4 != 0:
        return False
    elif year % 100 != 0:
        return True
    elif year % 400 != 0:
        return False
    else:
        return True
```

```
def last_day_of_month(year, month):
    if month <= 0 or month > 12:
        print('not a valid month')
        return -1
    if month == 2:
        if is_leap_year(year):
            return 29
        else:
            return 28
    elif (month <= 7) == (month % 2):
        return 31
    else:
        return 30
```


© MITB, SMU

Exercise 4-1: Ambiguity with i18n

- The ‘i18n’ method of shortening words discards too much information.
- This can lead to confusion, especially with words of the same length that start and end with the same pair of characters.

- Given a list of words, generate lists of words with the same ‘i18n’ acronym.
- Print each list with their ‘i18n’ acronym, e.g.,

```
i18n ['internationalization', 'institutionalization']
```


© MITB, SMU

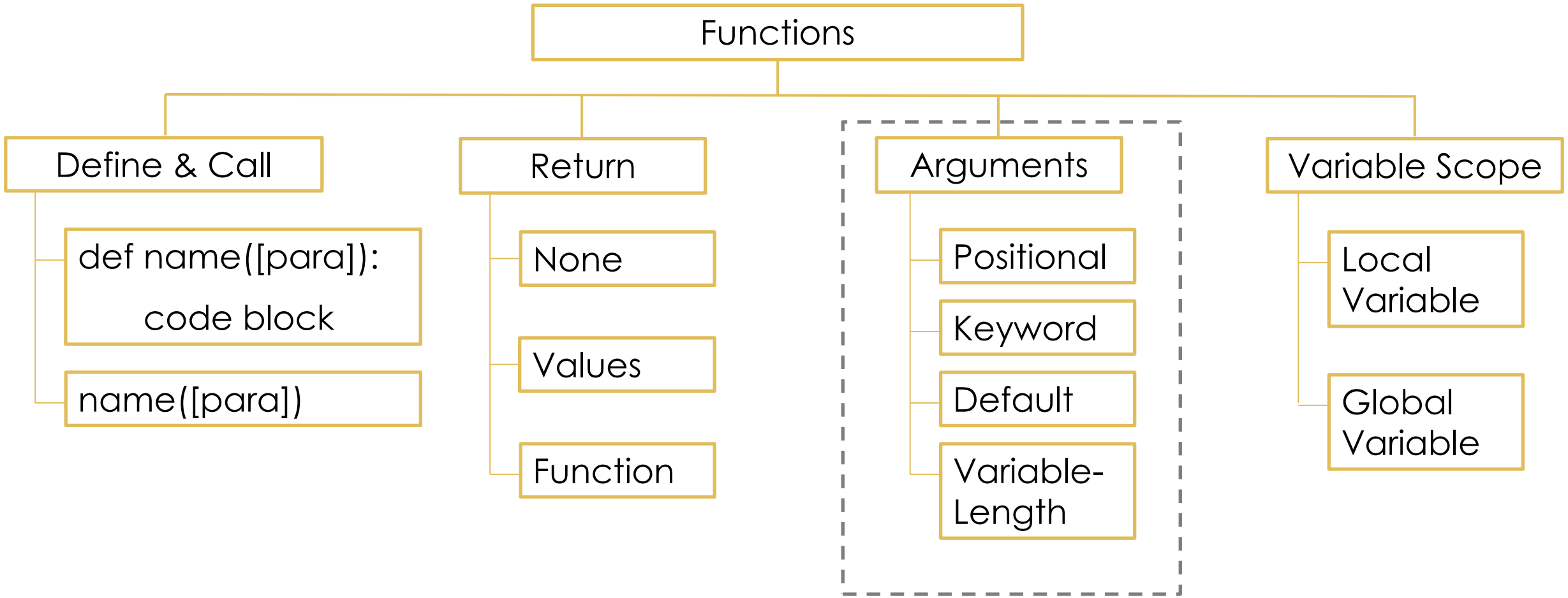
Exercise 4-2: Income Tax

- Implement a function to calculate personal income tax, as previously practiced during lecture 2.
- Generate the annual income for a group of 100 people, following normal distribution with mean 60,000 and standard deviation 10,000. You may use functions from library `random`.

- Calculate the personal income tax for each of the 100 people.
- Implement your own function to calculate the mean and standard deviation of the income tax of the 100 people without using library functions.

© MITB, SMU

Functions



© MITB, SMU

Positional Arguments

- The arguments passed to a function in correct positional order.
- The number of arguments in the function call should match exactly with the function definition.

```
def info(name, age):
    print(name, 'is', age, 'years old.')

info('Mike', 40)   # Mike is 40 years old.
info(40, 'Mike')   # 40 is Mike years old.    <- wrong positional order
info(40)           # error          <- number of arguments doesn't match
```


© MITB, SMU

Keyword Arguments

- The caller identifies the arguments by the parameter name.
- Can place arguments out of order.

```
def info(name, age):
    print(name, 'is', age, 'years old.')

info(name = 'Mike', age = 40)   # Mike is 40 years old.
info(age = 40, name = 'Mike')   # Mike is 40 years old.
```


© MITB, SMU

Default Arguments

confirm tested

- Assumes a default value if a value is **not** provided in the function call for that argument.
- Non-default argument is followed by default argument.

```
def func_name(non_default_1, non_default_2, default_1 = 1, default_2 = 2):
```

```
def info(name, age = 35):      # error if put info(age = 35, name)
    print(name, 'is', age, 'years old.')

info('Mike', 40)               # Mike is 40 years old.
info('Mike')                   # Mike is 35 years old.
```


© MITB, SMU

Variable-length arguments

- You may not know how many arguments there are while defining the function.
- We can pass a variable number of arguments to a function using special symbols.
- There are two special symbols:
  1. `*args` (Non-Keyword Arguments)
  2. `**kwargs` (Keyword Arguments)

© MITB, SMU

Variable-length arguments – *args

- With `*args`, any number of extra arguments can be stacked on to your current formal parameters (including zero extra arguments).
- The variable that we associate with the * becomes iterable.

```
def info(name, *args):
    hobby = []
    for a in args:
        hobby.append(a)
    print(name + "'s hobbies: " + ', '.join(hobby))
info('Mike')                          # Mike's hobbies:
info('Mike', 'hiking', 'reading')     # Mike's hobbies: hiking, reading
```

no star

see github for ex

© MITB, SMU

Variable-length arguments – **kwargs

- One can think of the `kwargs` as being a dictionary that maps each keyword to the value passed alongside it.

```
def info(name, **kwargs):
    hobby = []
    for k, v in kwargs.items():
        hobby.append(k + '-' + v)
    print(name + "'s hobbies: " + ', '.join(hobby))

info('Mike', first = 'hiking', second = 'reading')
# Mike's hobbies: first-hiking, second-reading
```


© MITB, SMU

Variable-length arguments – Example


```
def func(a1, a2, a3):
    print(a1, a2, a3)

args = ('Computational', 'Thinking', 'Python')
func(*args)     # Computational Thinking Python

kwargs = {'a1': 'Computational', 'a2': 'Thinking', 'a3': 'Python'}
func(**kwargs)  # Computational Thinking Python
```


© MITB, SMU

Variable-length arguments – Example

- `args` in front, followed by `kwargs`.

```
def func(*args, **kwargs):
    print('args: ', args)
    print('kwargs: ', kwargs)
    print(' '.join(args), ' '.join(kwargs.values()))

func('I', 'love', first = 'computational thinking', mid = 'and', last = 'Python')
# args: ('I', 'love')
# kwargs: {'first': 'computational thinking', 'mid': 'and', 'last': 'Python'}
# I love computational thinking and Python
```

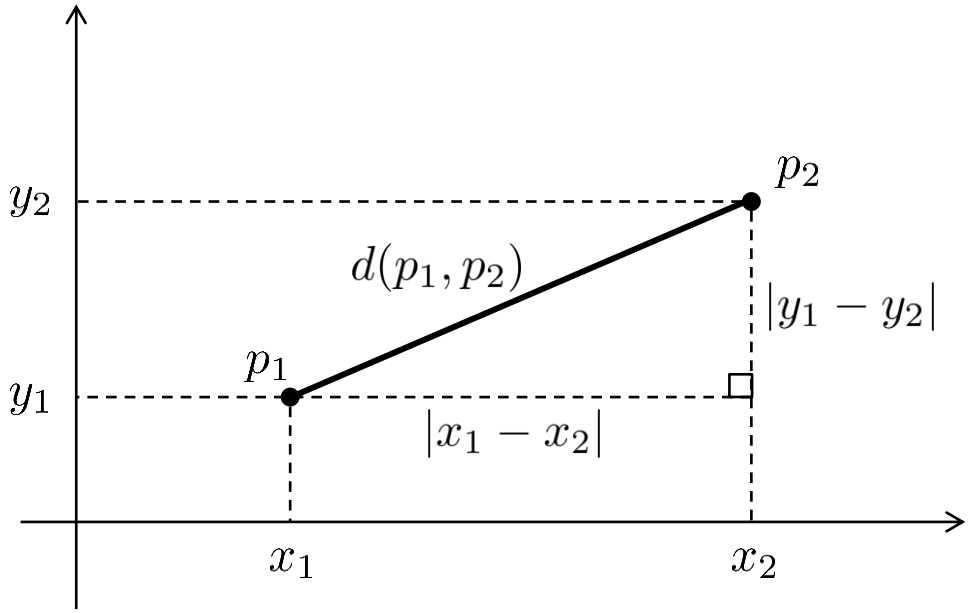

© MITB, SMU

Exercise 4-3: Euclidean Distance

- Euclidean distance between two points is easily calculated from the Cartesian coordinates of the points using the Pythagorean theorem.

```
def Euclid_dist(x1, y1, x2, y2):
    sum_sq = 0.0
    sum_sq += (x1 - x2) ** 2
    sum_sq += (y1 - y2) ** 2
    return sum_sq ** 0.5
```

$$d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$



What about Euclidean distance for two points in 3-dimensional space?

© MITB, SMU

Exercise 4-3: Euclidean Distance

- Multiple implementations of Euclidean distance have been discovered during a code review process, which differ in argument definition, but serving the same purpose.
- There are 7 ways of calling Euclidean distance, e.g.,

https://medium.com/mitb-for-all/writing-functions-with-flexibility-ebd0b99118f6

```
Euclid_dist(8, 5, 6, 2, 7, 9)
Euclid_dist(3, 8, 5, 6, 2, 7, 9)
Euclid_dist(3, [8, 5, 6], [2, 7, 9])
Euclid_dist(8, 5, 6, 2, 7, 9, dim = 3)
Euclid_dist([8, 5, 6], [2, 7, 9], dim = 3)
Euclid_dist(dim = 3, P1 = [8, 5, 6], P2 = [2, 7, 9])
Euclid_dist(P1 = [8, 5, 6], P2 = [2, 7, 9])
```

Are you able to write one `Euclid_dist` function that handles multiple types of arguments?

© MITB, SMU

Wooclap!

What is the output of the below code?

```
def fn(a, b = 0, *c, **d):
    print(a, b, c, d)

fn(1, 2, 3, 4, 5, x=6, y=7)
```

a. 1 0 (2, 3, 4, 5) {'x': 6, 'y': 7}

b. 1 2 (3, 4, 5) {'x': 6, 'y': 7}

c. 1 0 (2, 3, 4, 5, 6, 7) {}

d. 1 2 (3, 4, 5) {'b': 2, 'x': 6, 'y': 7}

Is function only about abstraction?

- Abstraction
  - Encapsulation
  - Reusability
  - Readability
- Decomposition
  - Modularity
  - Debugging and testing
  - Easy maintenance

© MITB, SMU

Global and Local Variables

Local Variable

- A variable declared **inside** the function's body (exists within the function's scope)
- Local variables are "cleared" once the function terminates
- For example, in the below code, `y` is a local variable

```
def test():
    y = 'local'
    print(y)

test()      # will print 'local'
print(y)    # NameError: name 'y' is not defined
```


© MITB, SMU

Global Variable

- A variable declared **outside** the function's body (exists within the global scope)
- Global variables are accessible anywhere in the code
- For example, in the below code, `y` is a global variable

```
y = 'global'
def test():
    global y
    print(y)

test()     # will print 'global'
print(y)   # will print 'global'
```


© MITB, SMU

Local vs. Global Variable

- The following examples demonstrate the difference between local and global variables

```
def f1():
    print(s)
    s = '456'
    print(s)

s = '123'
f1()
print(s)
```

```
def f2():
    s = '456'
    print(s)


s = '123'
f2()
print(s)
```

```
def f3():
    global s
    s = '456'
    print(s)

s = '123'
f3()
print(s)
```


© MITB, SMU

Lambda Expressions

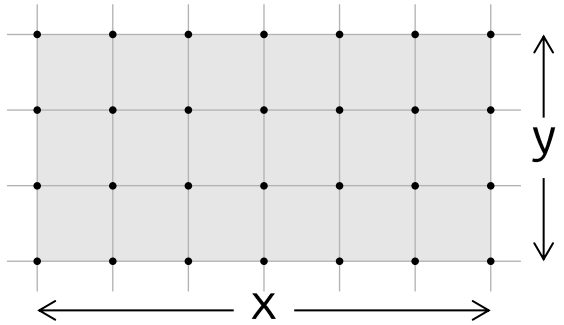
Lambda Expressions – Unnamed Functions

- Often used for short, throwaway functions that are not intended to be reused elsewhere
- **lambda argument_list: expression** → one-liner
- The structure of argument_list here is same as argument list in functions, i.e.
  - `a, b`
  - `a, b = 1, *args`
  - `**kwargs`
- expression must be single line, i.e.
  - `a + b`
  - `sum(a)`
  - `1 if a >10 else 0`

© MITB, SMU

Lambda Expressions – Characteristics

- It is anonymous.
- It has input and output.
- It usually performs simple tasks.




```
lambda x, y: (x + 1) * (y + 1)   # input: integers x, y
                                 # output: num of grid points on rectangle (x, y)

lambda *args: sum(args)          # input: any number of parameters
                                 # output: sum of the parameters

lambda x: 1.0 / len(x)           # input: string x
                                 # output: reciprocal of string length
```


© MITB, SMU

Lambda Expressions – Applications

1. Used together with list/dictionary comprehension.

2. Used together with `sorted()`, `filter()` and `map()`.

© MITB, SMU

List comprehension

- List / dictionary comprehension is an elegant way to create lists / dictionaries from ranges, strings or existing containers
- Compose a new list / dictionary with for loops

```
new_list = [expression(item) for item in an existing list]
```

- List Comprehension

```
a = [i**2 for i in range(3, 39, 2)]
```

- Dictionary Comprehension

```
b = {a[i]:i for i in range(len(a))}
```


© MITB, SMU

List comprehension – Examples

- Construct a list of random numbers

```
ls = [random.random() for i in range(20)]
```

- Construct a list of arithmetic sequence or geometric sequence

```
first_term, ratio = 512, 1.5

ls = [first_term * ratio ** i for i in range(20)]

print(ls)
```


© MITB, SMU

Wooclap!

What is the length of the list constructed by:

```
ls = [ i**2 for i in range(5, 10) ]
```

a. 5
b. 6
c. 9
d. 10

Exercise 4-4: Affordability in buying food

- The price of Waterloo street chicken rice increases at 5% yearly.
- Ah Huat's annual salary increases by only $500.

- Task: to show that the affordability will be halved in 20 years' time, in terms of the number of chicken rice that can be purchased.

© MITB, SMU

SMU Classification: Restricted

Exercise 4-5: Convert temperatures with lambda expression

- Given a list of temperatures in Fahrenheit
- Construct a list of corresponding temperatures in Celsius, by converting each temperature from Fahrenheit to Celsius with a lambda expression and the formula below:

$$°C = (°F - 32) \times \frac{5}{9}$$

© MITB, SMU

Lambda with sorted, filter and map

- `sorted(iterable, key = None, reverse = False)`
  - sorts the elements in the given iterable by key

```
sorted([11, 2, -7, -1, 5], key = lambda x: abs(x))  # [-1, 2, 5, -7, 11]
```

- `filter(function, iterable)`
  - filters the given iterable with the help of the given function

```
list(filter(lambda n: n % 2 == 1, [1, 2, 3, 4, 5]))  # [1, 3, 5]
```

- `map(function, iterable, ...)`
  - applies a given function to each item of the given iterable

```
list(map(lambda x: x + 1, [1, 2, 3]))                # [2, 3, 4]
```


© MITB, SMU

Wooclap!

What is the output of the below code?

```
ls = [3, 2, 1, 5, 7]
ls = sorted(ls, key = lambda x: x % 4)
print(ls)
```

a. [3, 2, 1, 5, 7]

b. [1, 5, 2, 3, 7]

c. [1, 1, 2, 3, 3]

d. [1, 2, 3, 5, 7]

Exercise 4-6: Physicists' schedule

- At a conference, each speaker is given 25 minutes. They starts at 8:05am, and present in the ascending order of their last name.
- Print the schedule of the presentations.

© MITB, SMU

SMU
SINGAPORE MANAGEMENT UNIVERSITY

School of
Computing and
Information Systems

Questions?

Thank You

